



UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS

Facultad de Ingeniería

Object-Oriented Programming

SYNKRO

Presented by:

Kevin Navarrete

Valeria Usma

Instructor:

Carlos Andrés Sierra

1. Introduction

Instant communication is a necessity in today's world. This project seeks to create an efficient application that enables secure and accessible user interaction, inspired by the basic functions of Skype. The application will allow users to register, log in, send private and group messages, share files, manage contacts, and customize their profile.

The project seeks to offer a practical and functional communication solution while applying the fundamental concepts of object-oriented programming. The main motivation is to design a flexible, organized, and scalable system with well-structured classes and relationships that facilitate future growth and maintenance.

2. Objectives

The project aims to develop an application that allows users to:

- Send and receive messages.
- Make voice and video calls.
- Implement an authentication system so that each user has a personal account.
- Create an intuitive and easy-to-use interface.

Specific Objectives

- Implement a secure authentication system.
- Develop private and group messaging functionality.
- Manage public and private groups.
- Allow contact management and profile customization.
- Apply SOLID principles of object-oriented programming.
- Model the system with cohesive classes and properly structured relationships.
- Implement a flexible, scalable, and maintainable design.

3. Requirements

Functional Requirements

1. Registration and login using email and password.
2. Sending and receiving private messages between users.
3. Invitation system for friends and groups.
4. Making voice calls.
5. Creation and management of public and private groups.
6. Contact management and profile customization.
7. Account control: temporary suspension and deletion.

Non-Functional Requirements

1. Protection of user data and communications.
2. Prevention of duplicate usernames and email addresses.
3. Immediate application of account changes.
4. Support for multiple users without affecting platform stability.

Although the model includes voice and video calling functionality, this feature will not be implemented in the current version of the project due to time and scope constraints. The system design allows for the possibility of adding this functionality in future versions, ensuring that its integration is compatible with the current architecture.

4. User Stories

Story: User Registration (Priority: High)

User Story: As a new user, I want to register with an email, a unique username, and a password.

Acceptance Criteria: The user must enter valid and unique data. The system displays an error if the email or name already exists. The user is automatically logged in after registration.

Story: Private Communication (Priority: High)

User Story: As a user, I want to send messages with other users.

Acceptance Criteria: Privacy and encryption must be guaranteed. Messages are only accessible to participants.

Story: Group Messaging (Priority: High)

User Story: As a user, I want to send messages to groups and receive instant replies.

Acceptance Criteria: Messages are displayed instantly with names and timestamps. Participants can respond immediately.

Story: Group Management (Priority: High)

User Story: As a user, I want to create or delete groups.

Acceptance Criteria: Only the creator or administrators can delete groups. Deletion clears all group history.

Story: Member Management (Priority: High)

User Story: As a user, I want to add or remove members from a group I manage.

Acceptance Criteria: Confirmation is required when deleting members. Administrators can manage the participant list.

Story: Temporary Suspension (Priority: High)

User Story: As a user, I want to be able to temporarily close my account.

Acceptance Criteria: The deactivated account is invisible to other users and can be reactivated upon login.

Story: Joining Groups (Priority: Medium)

User Story: As a user, I want to join existing groups.

Acceptance Criteria: Users can search for and join public groups, or request access to private groups.

Story: Creating Groups (Priority: Medium)

User Story: As a user, I want to create public or private groups.

Acceptance Criteria: The creator decides the group's privacy and becomes the initial administrator.

Story: Group Management (Priority: Medium)

User Story: As a user, I want only administrators to manage groups.

Acceptance Criteria: The administrator can assign and revoke permissions to other users.

Story: Interface Customization (Priority: Low)

User Story: As a user, I want to customize the app's appearance.

Acceptance Criteria: Users can switch between light/dark modes, modify backgrounds, and font sizes. Changes are automatically saved.

Story: Voice Calls (Priority: Low)

User Story: As a user, I want to make voice calls with my contacts to communicate directly, but this feature will be considered for future versions due to the current scope of the project.

5. Mock-Ups

Synkro

Correo electronico

email.domain.com

Contraseña

password

Iniciar sesion

[Olvido su contraseña?](#)

No tiene una cuenta? [Cree una](#)

Synkro

Crea una cuenta

Ingresa tu correo electronico para crear una cuenta

email@domain.com

Crear cuenta con correo electronico

o continuar con



Google

Haciendo clic en continuar, aceptas nuestros
terminos de Servicio y Política de privacidad



Helena Hills

Active 20m ago

📞

🗨

No honestly I'm thinking of a career pivot

This is the main chat template

Nov 30, 2023, 9:41 AM

Oh?

Cool

How does it work?

Simple

You just edit any text to type in the conversation you want to show, and delete any bubbles you don't want to use

Boom

Hmmm

I think I get it

Will head to the Help Center if I have more questions tho

Enter your message

🎤

😊

📎

Synkro

Correo electronico

Contraseña

Iniciar sesion

[Olvido su contraseña?](#)

No tiene una cuenta? [Cree una](#)

Synkro

🔍

Buscar Chats



Helena Hills

Will head to the Help Center...



Carlo Emilio

Let's go



Oscar Davis

Trueeeeeee



Daniel Jay Park

lol yeah, are you coming to the lunc...



Mark Rojas

great catching up over dinner!!



Giannis Constantinou

yep 🍷🍷



Briana Lewis

When are you coming back to town?...



Mom

Thanks!



Helena

Active 20m ago

View profile

🔍

Search chat

📎

Sent images

⋮

More options

6. CRC Cards

1. Class: User

Responsibilities:

- Manage profile and contacts.
- Participate in conversations.
- Upload files.
- Make calls.

Contributors:

- Authentication Service
 - Message Sender
 - Call
 - Call History
 - File
-

2. Class: Authentication Service

Responsibilities:

- Register users.
- Manage logins and logouts.

Contributors:

- User
-

Interface: Message Sender

Responsibilities:

- Send messages.

Contributors:

- Conversation
 - User
-

3. Class: Conversation

Responsibilities:

- Manage messages.
- Store participants.

Collaborators:

- Message
 - File
 - Channel
 - Direct Message
-

4. Class: Channel

Responsibilities:

- Manage public or private groups.
- Add or remove participants.

Collaborators:

- Conversation
-

5. Class: Direct-Message

Responsibilities:

- Manage one-on-one chats.
- Block or unblock users.

Collaborators:

- Conversation
-

6. Class: Message

Responsibilities:

- Store text or files.
- Edit and delete messages.

Contributors:

- File
 - Conversation
-

7. Clase: File

Responsibilities:

- Manage files.
- Allow downloads.

Contributors:

- Image
 - Doc
 - User
 - Conversation
-

8. Clase: Call

Responsibilities:

- Initiate and end voice calls.
- Calculate call duration.

Contributors:

- User
 - Call History
-

9. Class: Call History

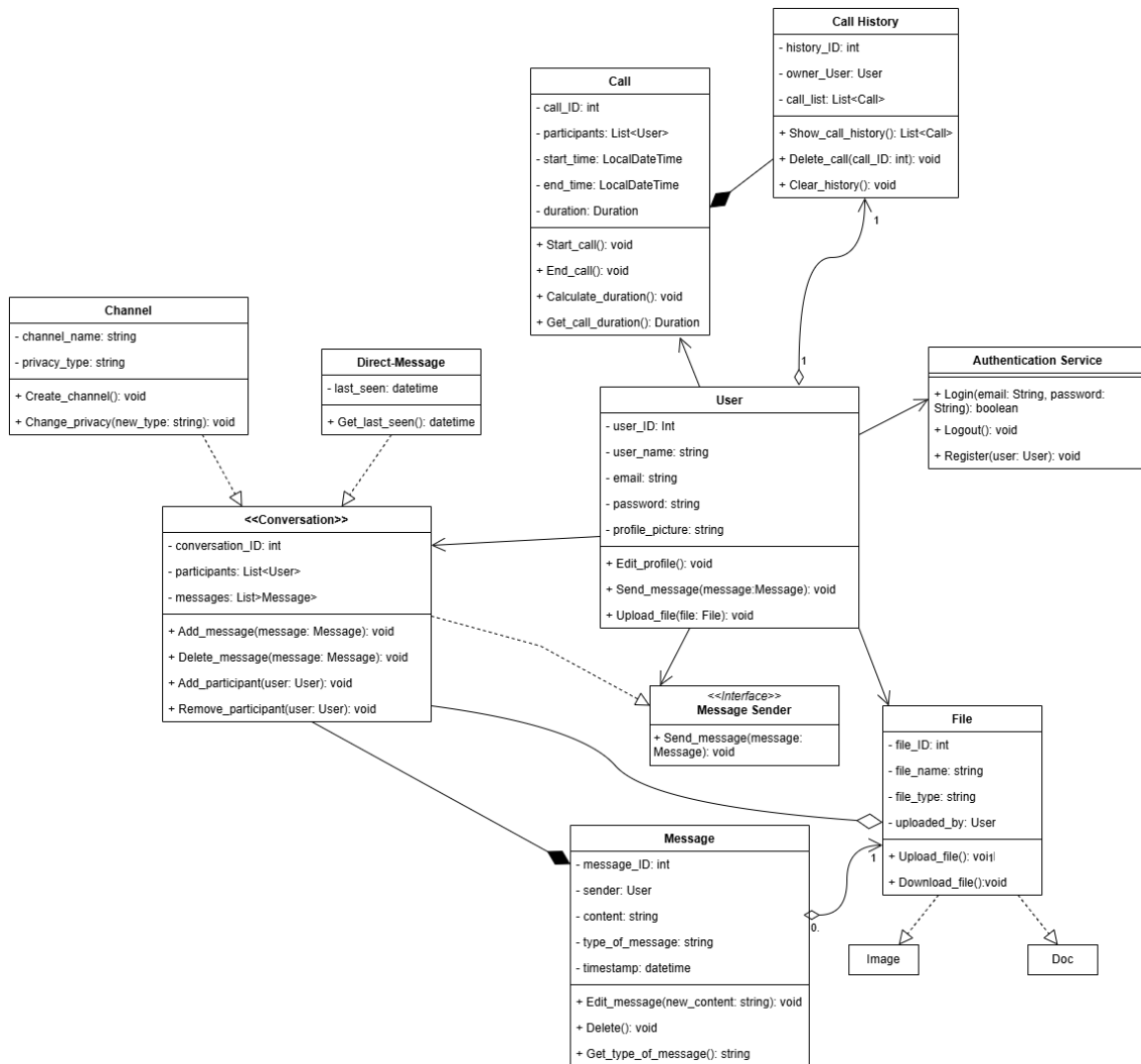
Responsibilities:

- Record user call history.
- Delete calls.

Contributors:

- Call
- User

7. UML Diagram



Main modeling decisions:

1. **User:** Manages the profile, contacts, and calls. It does not handle authentication directly.
2. **Authentication Service:** Exclusive class for registration, login, and logout.
3. **Conversation:** General class that organizes chats.
4. **Channel and DirectMessage:** Subclasses of Conversation, allowing differentiation between group and private chats.
5. **Message:** An object that represents each message; it can contain text, files, or both.
6. **File:** Parent class for different file types (Image, Doc). It allows extending the system without modifying it.
7. **Call and CallHistory:** Model of voice calls and their history per user.
8. **Message Sender:** Interface that ensures message sending without being directly coupled to the conversation type.

The AuthenticationService class was created so that authentication could be managed independently. The User class is limited to storing profile data and managing contacts, ensuring that each class has a single responsibility.

Furthermore, the system allows adding new file types (for example, images or documents) without modifying existing classes. File is an extensible class, which facilitates future extensions.

Furthermore, the Channel and DirectMessage classes are subtypes of Conversation and can be used where a conversation is expected without affecting the system logic.

Finally, the MessageSender interface was created so that classes that send messages are not forced to implement methods that do not correspond to them. User only interacts with this interface and does not depend on specific implementations.

The User class depends on the IMessageSender abstraction, which allows changing the class that manages message sending without affecting the User structure. This decoupling ensures flexibility in the system.

The application follows a layered architecture pattern to keep the code organized, decoupled, and easy to maintain. It includes the following layers:

- **Presentation layer:** Contains GUI classes such as Login and Register, developed using Java Swing.
- **Business logic layer:** Includes the core classes that manage the application's behavior, such as User, Message, Call, and CallHistory.
- **Data persistence layer:** Includes classes like UserStorage, responsible for saving and retrieving data using JSON files.

8. How do we apply OOP concepts

For the project implementation, each class was designed to represent real-world objects, such as users, messages, groups, calls, and files. The relationships between classes were modeled to reflect their interactions and dependencies, allowing each object to maintain its own state and behaviors.

Encapsulation: The attributes of each class were declared private to protect data integrity, and can only be accessed or modified through public methods (getters and setters).

Inheritance: Hierarchies were created, such as the Conversation class, which is general and specializes in Channel and DirectMessage, allowing reuse of common code and behaviors.

Polymorphism: Thanks to inheritance and the implementation of the IMessageSender interface, classes can be used interchangeably, allowing for flexible handling of different

types of conversations. Composition and Aggregation: Composition relationships were established, such as Conversation and Message, where messages do not exist outside the conversation, and aggregation relationships, such as the association between Conversation and File, where files can exist independently.

Abstraction: Classes such as File and interfaces such as IMessageSender were created to encapsulate general behaviors, allowing other classes to interact with them without knowing the internal details of their implementation..

9. Code

1. Login Interface

```
package View;

// Importación de librerías necesarias para la interfaz gráfica y eventos
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.border.LineBorder;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.FocusAdapter;
import java.awt.event.FocusEvent;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import Classes.UserStorage;
import org.json.JSONObject;

/**
 * Clase Login que extiende JFrame.
 * Representa la ventana principal de inicio de sesión de la aplicación.
 */
public class Login extends JFrame {
    private JTextField emailField;
    private JPasswordField passwordField;
    private JButton loginButton;
    private JButton registerButton;
    private JCheckBox rememberCheckBox;

    // Colores principales personalizados para la interfaz
    private final Color SYNKRO = new Color(180, 66, 235);
    private final Color BACKGROUND_COLOR = new Color(255, 255, 255);
    private final Color FIELD_COLOR = Color.WHITE;
```

```

// Constructor de la ventana Login
public Login() {
    initializeLoginFrame();          // Configura las propiedades básicas de la ventana
    createLoginComponents();         // Crea los componentes visuales
    setupLoginLayout();              // Organiza los componentes en el panel
    addLoginEventListeners();        // Añade la lógica de eventos (botones, campos, etc.)
}

// Configura las propiedades de la ventana principal
private void initializeLoginFrame() {
    setTitle("Synkro - Iniciar Sesión");
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setSize(600, 600);
    setLocationRelativeTo(null);
    setResizable(false);
    getContentPane().setBackground(BACKGROUND_COLOR);
}

// Crea los campos, botones y checkbox del formulario de inicio de sesión
private void createLoginComponents() {
    // Email field con placeholder
    emailField = createStyledTextField("Email o teléfono");

    // Password field
    passwordField = new JPasswordField();
    passwordField.setFont(new Font("Segoe UI", Font.PLAIN, 14));
    passwordField.setBorder(BorderFactory.createCompoundBorder(
        new LineBorder(Color.LIGHT_GRAY, 1),
        new EmptyBorder(10, 15, 10, 15)
    ));
    passwordField.setPreferredSize(new Dimension(300, 45));

    // Botones
    loginButton = createStyledButton("Iniciar sesión", SYNKRO);
    registerButton = createStyledButton("Crear cuenta", Color.WHITE, SYNKRO);
}

```

```

// Checkbox
rememberCheckBox = new JCheckBox("Recordarme");
rememberCheckBox.setFont(new Font("Segoe UI", Font.PLAIN, 12));
rememberCheckBox.setBackground(BACKGROUND_COLOR);
rememberCheckBox.setForeground(Color.DARK_GRAY);
}

// Crea un campo de texto estilizado con placeholder
private JTextField createStyledTextField(String placeholder) {
    JTextField field = new JTextField();
    field.setFont(new Font("Segoe UI", Font.PLAIN, 14));
    field.setBorder(BorderFactory.createCompoundBorder(
        new LineBorder(Color.LIGHT_GRAY, 1),
        new EmptyBorder(10, 15, 10, 15)
    ));
    field.setPreferredSize(new Dimension(300, 45));

    // Efecto placeholder
    setupPlaceholder(field, placeholder);

    return field;
}

// Establece el comportamiento del placeholder en campos de texto
private void setupPlaceholder(JTextField field, String placeholder) {
    field.setText(placeholder);
    field.setForeground(Color.GRAY);

    field.addFocusListener(new FocusAdapter() {
        @Override
        public void focusGained(FocusEvent e) {
            if (field.getText().equals(placeholder)) {
                field.setText("");
                field.setForeground(Color.BLACK);
            }
        }
    })
}

```

```

        @Override
        public void focusLost(FocusEvent e) {
            if (field.getText().isEmpty()) {
                field.setText(placeholder);
                field.setForeground(Color.GRAY);
            }
        }
    });
}

// Crea un botón estilizado con color de fondo predeterminado
private JButton createStyledButton(String text, Color bgColor) {
    return createStyledButton(text, bgColor, Color.WHITE);
}

// Crea un botón estilizado con color de fondo y texto personalizado
private JButton createStyledButton(String text, Color bgColor, Color textColor) {
    JButton button = new JButton(text);
    button.setFont(new Font("Segoe UI", Font.BOLD, 14));
    button.setBackground(bgColor);
    button.setForeground(textColor);
    button.setBorder(new EmptyBorder(12, 20, 12, 20));
    button.setFocusPainted(false);
    button.setCursor(new Cursor(Cursor.HAND_CURSOR));
    button.setPreferredSize(new Dimension(300, 45));
    button.setContentAreaFilled(true);
    button.setOpaque(true);
    button.setBorderPainted(false);
}

```

```

// Efecto hover al pasar el mouse sobre el botón
button.addMouseListener(new MouseAdapter() {
    @Override
    public void mouseEntered(MouseEvent e) {
        if (bgColor.equals(SYNKRO)) {
            button.setBackground(SYNKRO.darker());
        } else {
            button.setBackground(new Color(245, 245, 245));
        }
    }

    @Override
    public void mouseExited(MouseEvent e) {
        button.setBackground(bgColor);
    }
});

return button;
}

// Organiza y distribuye visualmente los componentes en el panel principal
private void setupLoginLayout() {
    setLayout(new BorderLayout());

    // Panel principal
    JPanel mainPanel = new JPanel();
    mainPanel.setLayout(new BoxLayout(mainPanel, BoxLayout.Y_AXIS));
    mainPanel.setBackground(BACKGROUND_COLOR);
    mainPanel.setBorder(new EmptyBorder(40, 50, 40, 50));

    JLabel subtitleLabel = new JLabel("Inicia sesión en tu cuenta");
    subtitleLabel.setFont(new Font("Segoe UI", Font.PLAIN, 16));
    subtitleLabel.setForeground(new Color(100, 100, 100));
    subtitleLabel.setAlignmentX(Component.CENTER_ALIGNMENT);

```

```

// Cargar y redimensionar la imagen
ImageIcon icon = new ImageIcon(getClass().getResource("/Images/Logo.png"));
Image image = icon.getImage().getScaledInstance(200, 100, Image.SCALE_SMOOTH);
JLabel imageLabel = new JLabel(new ImageIcon(image));
imageLabel.setAlignmentX(Component.CENTER_ALIGNMENT);
// Agregar imagen al panel principal
mainPanel.add(imageLabel);
mainPanel.add(Box.createVerticalStrut(20)); // Espacio debajo de la imagen

// Espaciado y componentes
mainPanel.add(subtitleLabel);
mainPanel.add(Box.createVerticalStrut(40));
mainPanel.add(emailField);
mainPanel.add(Box.createVerticalStrut(15));
mainPanel.add(passwordField);
mainPanel.add(Box.createVerticalStrut(10));

// Panel para checkbox y "¿Olvidaste tu contraseña?"
JPanel optionsPanel = new JPanel(new BorderLayout());
optionsPanel.setBackground(BACKGROUND_COLOR);
optionsPanel.add(rememberCheckBox, BorderLayout.WEST);

JLabel forgotLabel = new JLabel("¿Olvidaste tu contraseña?");
forgotLabel.setFont(new Font("Segoe UI", Font.PLAIN, 12));
forgotLabel.setForeground(SYMKRO);
forgotLabel.setCursor(new Cursor(Cursor.HAND_CURSOR));
optionsPanel.add(forgotLabel, BorderLayout.EAST);

mainPanel.add(optionsPanel);
mainPanel.add(Box.createVerticalStrut(25));
loginButton.setAlignmentX(Component.CENTER_ALIGNMENT);
mainPanel.add(loginButton);
mainPanel.add(Box.createVerticalStrut(15));

// Separador
JPanel separatorPanel = new JPanel(new BorderLayout());
separatorPanel.setBackground(BACKGROUND_COLOR);
separatorPanel.add(new JSeparator(), BorderLayout.CENTER);
JLabel orLabel = new JLabel(" o ");
orLabel.setBackground(BACKGROUND_COLOR);
orLabel.setOpaque(true);
separatorPanel.add(orLabel, BorderLayout.CENTER);

mainPanel.add(separatorPanel);
mainPanel.add(Box.createVerticalStrut(15));
registerButton.setAlignmentX(Component.CENTER_ALIGNMENT);
mainPanel.add(registerButton);

// Scroll para el panel principal
JScrollPane scrollPane = new JScrollPane(mainPanel, JScrollPane.VERTICAL_SCROLLBAR_AS_NEEDED, JScrollPane.HORIZONTAL_SCROLLBAR_NEVER);
scrollPane.setBorder(null);
scrollPane.getVerticalScrollBar().setUnitIncrement(16); // suaviza el scroll
add(scrollPane, BorderLayout.CENTER);
}

// Añade los listeners para los botones y eventos importantes
private void addLoginEventListeners() {
    loginButton.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            performLogin();
        }
    });
}

registerButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        openRegisterFrame();
    }
});
}

```



```

        // Permitir login con Enter
        getRootPane().setDefaultButton(loginButton);
    }

    // Lógica principal de autenticación del usuario
    private void performLogin() {
        String email = emailField.getText();
        String password = new String(passwordField.getPassword());

        // Validaciones básicas
        if (email.equals("Email o teléfono") || email.isEmpty()) {
            showError("Por favor ingresa tu email o teléfono");
            return;
        }

        if (password.isEmpty()) {
            showError("Por favor ingresa tu contraseña");
            return;
        }

        // Validar credenciales
        if (authenticateUser(email, password)) {
            JOptionPane.showMessageDialog(this,
                ";Inicio de sesión exitoso!\nBienvenido: " + email,
                "Éxito", JOptionPane.INFORMATION_MESSAGE);
        } else {
            showError("Email o contraseña incorrectos");
        }
    }

    // Método que valida los datos ingresados consultando en UserStorage
    private boolean authenticateUser(String email, String password) {
        JSONObject user = UserStorage.findUserByEmail(email);
        if (user != null && user.getString("contraseña").equals(password)) {
            return true;
        }
        return false;
    }

    // Abre la ventana de registro y oculta la actual
    private void openRegisterFrame() {
        new Register().setVisible(true);
        this.setVisible(false);
    }

    // Muestra un mensaje emergente de error
    private void showError(String message) {
        JOptionPane.showMessageDialog(this, message, "Error", JOptionPane.ERROR_MESSAGE);
    }
}

```

1.1 Login Screen



Inicia sesión en tu cuenta

☐ Recordarme

[¿Olvidaste tu contraseña?](#)

Iniciar sesión

o

Crear cuenta

2. Register Interface

```
package View;

// Importaciones necesarias para la interfaz y eventos
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.border.LineBorder;
import java.awt.*;
import java.awt.event.FocusAdapter;
import java.awt.event.FocusEvent;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import Classes.UserStorage;
import org.json.JSONObject;

/**
 * Clase Register que extiende JFrame.
 * Representa la ventana de registro de usuario para crear una nueva cuenta en la aplicación.
 */
public class Register extends JFrame {
    // Campos de texto y botones de la interfaz
    private JTextField nameField, emailField, phoneField;
    private JPasswordField passwordField, confirmPasswordField;
    private JButton registerButton, backButton;
    private JCheckBox termsCheckBox;
    private JComboBox<String> countryComboBox;

    // Paleta de colores coherente con el Login
    private final Color SYNKRO = new Color(180, 66, 235);
    private final Color BACKGROUND_COLOR = new Color(255, 255, 255);

    // Constructor principal
    public Register() {
        initializeRegisterFrame(); // Configura propiedades básicas de la ventana
        createRegisterComponents(); // Crea los elementos visuales
        setupRegisterLayout(); // Organiza los componentes
        addRegisterEventListeners(); // Añade acciones a botones y campos
    }

    // Configura la ventana principal del registro
    private void initializeRegisterFrame() {
        setTitle("Synkro - Crear Cuenta");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(450, 650);
        setLocationRelativeTo(null);
        setResizable(false);
        getContentPane().setBackground(BACKGROUND_COLOR);
    }

    // Crea y configura los elementos visuales del formulario
    private void createRegisterComponents() {
        // Campos de texto
        nameField = createStyledTextField("Nombre completo");
        emailField = createStyledTextField("Email");
        phoneField = createStyledTextField("Teléfono (opcional)");

        // Menú desplegable con países
        String[] countries = {"Colombia", "México", "España", "Argentina", "Chile", "Perú"};
        countryComboBox = new JComboBox<>(countries);
        countryComboBox.setFont(new Font("Segoe UI", Font.PLAIN, 14));
        countryComboBox.setPreferredSize(new Dimension(300, 45));

        // Campos para contraseña y confirmación
        passwordField = createStyledPasswordField();
        confirmPasswordField = createStyledPasswordField();

        // Botones
        registerButton = createStyledButton("Crear cuenta", SYNKRO);
        backButton = createStyledButton("Volver", Color.WHITE, SYNKRO);

        // Checkbox de aceptación de términos
        termsCheckBox = new JCheckBox("<html>Acepto los <a href='#'>términos de servicio</a> y la <a href='#'>política de privacidad</a></html>");
        termsCheckBox.setFont(new Font("Segoe UI", Font.PLAIN, 11));
        termsCheckBox.setBackground(BACKGROUND_COLOR);
    }
}
```

```
// Crea un campo de texto con estilo personalizado y placeholder
private JTextField createStyledTextField(String placeholder) {
    JTextField field = new JTextField();
    field.setFont(new Font("Segoe UI", Font.PLAIN, 14));
    field.setBorder(BorderFactory.createCompoundBorder(
        new LineBorder(Color.LIGHT_GRAY, 1),
        new EmptyBorder(10, 15, 10, 15)
    ));
    field.setPreferredSize(new Dimension(300, 45));

    // Configura el placeholder
    field.setText(placeholder);
    field.setForeground(Color.GRAY);

    field.addFocusListener(new FocusAdapter() {
        @Override
        public void focusGained(FocusEvent e) {
            if (field.getText().equals(placeholder)) {
                field.setText("");
                field.setForeground(Color.BLACK);
            }
        }

        @Override
        public void focusLost(FocusEvent e) {
            if (field.getText().isEmpty()) {
                field.setText(placeholder);
                field.setForeground(Color.GRAY);
            }
        }
    });

    return field;
}
```

```

// Crea un campo de contraseña estilizado
private JPasswordField createStyledPasswordField() {
    JPasswordField field = new JPasswordField();
    field.setFont(new Font("Segoe UI", Font.PLAIN, 14));
    field.setBorder(BorderFactory.createCompoundBorder(
        new LineBorder(Color.LIGHT_GRAY, 1),
        new EmptyBorder(10, 15, 10, 15)
    ));
    field.setPreferredSize(new Dimension(300, 45));
    return field;
}

// Crea botón estilizado con fondo por defecto blanco
private JButton createStyledButton(String text, Color bgColor) {
    return createStyledButton(text, bgColor, Color.WHITE);
}

// Crea botón estilizado con fondo y texto personalizados
private JButton createStyledButton(String text, Color bgColor, Color textColor) {
    JButton button = new JButton(text);
    button.setFont(new Font("Segoe UI", Font.BOLD, 14));
    button.setBackground(bgColor);
    button.setForeground(textColor);
    button.setBorder(new EmptyBorder(12, 20, 12, 20));
    button.setFocusPainted(false);
    button.setCursor(new Cursor(Cursor.HAND_CURSOR));
    button.setPreferredSize(new Dimension(300, 45));
    button.setContentAreaFilled(true);
    button.setOpaque(true);
    button.setBorderPainted(false);
}

```

```

// Efecto hover al pasar el mouse
button.addMouseListener(new MouseAdapter() {
    @Override
    public void mouseEntered(MouseEvent e) {
        if (bgColor.equals(SYNKRO)) {
            button.setBackground(SYNKRO.darker());
        } else {
            button.setBackground(new Color(245, 245, 245));
        }
    }

    @Override
    public void mouseExited(MouseEvent e) {
        button.setBackground(bgColor);
    }
});

return button;
}

// Organiza los componentes visuales en el panel principal
private void setupRegisterLayout() {
    setLayout(new BorderLayout());

    JPanel mainPanel = new JPanel();
    mainPanel.setLayout(new BoxLayout(mainPanel, BoxLayout.Y_AXIS));
    mainPanel.setBackground(BACKGROUND_COLOR);
    mainPanel.setBorder(new EmptyBorder(30, 50, 30, 50));

    // Título y subtítulo
    JLabel titleLabel = new JLabel("Crear cuenta");
    titleLabel.setFont(new Font("Segoe UI Light", Font.PLAIN, 28));
    titleLabel.setForeground(SYNKRO);
    titleLabel.setAlignmentX(Component.CENTER_ALIGNMENT);

```

```

// Efecto hover al pasar el mouse
button.addMouseListener(new MouseAdapter() {
    @Override
    public void mouseEntered(MouseEvent e) {
        if (bgColor.equals(SYNKRO)) {
            button.setBackground(SYNKRO.darker());
        } else {
            button.setBackground(new Color(245, 245, 245));
        }
    }

    @Override
    public void mouseExited(MouseEvent e) {
        button.setBackground(bgColor);
    }
});

return button;
}

// Organiza los componentes visuales en el panel principal
private void setupRegisterLayout() {
    setLayout(new BorderLayout());

    JPanel mainPanel = new JPanel();
    mainPanel.setLayout(new BoxLayout(mainPanel, BoxLayout.Y_AXIS));
    mainPanel.setBackground(BACKGROUND_COLOR);
    mainPanel.setBorder(new EmptyBorder(30, 50, 30, 50));

    // Título y subtítulo
    JLabel titleLabel = new JLabel("Crear cuenta");
    titleLabel.setFont(new Font("Segoe UI Light", Font.PLAIN, 28));
    titleLabel.setForeground(SYNKRO);
    titleLabel.setAlignmentX(Component.CENTER_ALIGNMENT);

```

```

// Crea un panel con etiqueta y componente alineados verticalmente
private JPanel createLabeledField(String label, JComponent component) {
    JPanel panel = new JPanel();
    panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));
    panel.setBackground(BACKGROUND_COLOR);
    panel.setAlignmentX(Component.CENTER_ALIGNMENT);

    JLabel jLabel = new JLabel(label);
    jLabel.setFont(new Font("Segoe UI", Font.PLAIN, 12));
    jLabel.setForeground(Color.DARK_GRAY);
    jLabel.setAlignmentX(Component.LEFT_ALIGNMENT);

    component.setAlignmentX(Component.LEFT_ALIGNMENT);

    panel.add(jLabel);
    panel.add(Box.createVerticalStrut(5));
    panel.add(component);
    panel.add(Box.createVerticalStrut(15));

    return panel;
}

// Añade la lógica a los botones
private void addRegisterEventListeners() {
    registerButton.addActionListener(e -> performRegistration());
    backButton.addActionListener(e -> {
        new Login().setVisible(true);
        this.dispose();
    });

    getRootPane().setDefaultButton(registerButton);
}

```



```
// Método que valida los datos, registra el usuario y vuelve al login
private void performRegistration() {
    // Validaciones
    if (!validateRegistrationFields()) {
        return;
    }
    // Si pasa la validación, guardar el usuario:
    String name = nameField.getText();
    String email = emailField.getText();
    String password = new String(passwordField.getPassword());

    UserStorage.saveUser(name, email, password);

    // Simular registro exitoso
    JOptionPane.showMessageDialog(this,
        ";Registro exitoso!\nPuedes iniciar sesión con tu nueva cuenta.",
        "Éxito", JOptionPane.INFORMATION_MESSAGE);

    // Volver al login
    new Login().setVisible(true);
    this.dispose();
}

// Realiza validaciones sobre los campos de entrada del formulario
private boolean validateRegistrationFields() {
    String name = nameField.getText();
    String email = emailField.getText();
    String password = new String(passwordField.getPassword());
    String confirmPassword = new String(confirmPasswordField.getPassword());

    if (name.equals("Nombre completo") || name.trim().isEmpty()) {
        showError("El nombre es obligatorio");
        return false;
    }
}
```

```

        if (email.equals("Email") || email.trim().isEmpty()) {
            showError("El email es obligatorio");
            return false;
        }

        if (!email.contains("@") || !email.contains(".")) {
            showError("Por favor ingresa un email válido");
            return false;
        }

        if (password.length() < 6) {
            showError("La contraseña debe tener al menos 6 caracteres");
            return false;
        }

        if (!password.equals(confirmPassword)) {
            showError("Las contraseñas no coinciden");
            return false;
        }

        if (!termsCheckBox.isSelected()) {
            showError("Debes aceptar los términos y condiciones");
            return false;
        }

        return true;
    }

    // Muestra mensaje emergente de error
    private void showError(String message) {
        JOptionPane.showMessageDialog(this, message, "Error", JOptionPane.ERROR_MESSAGE);
    }
}

```

2.1 Register Screen

Nombre completo *

Email *

País

Teléfono

Contraseña *

Confirmar contraseña *

☐ Acepto los [términos de servicio](#) y la [política de privacidad](#)

Crear cuenta

3. Main App

```
package Main;

import View.Login;
import javax.swing.*.*;

public class App {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            try {
                UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
            } catch (Exception e) {
                e.printStackTrace();
            }
            new Login().setVisible(true);
        });
    }
}
```

4. Json

```
[{
  "correo": "valeue11@gmail.com",
  "nombre": "Valeria",
  "contraseña": "Vale0797"
}]
```

Instead of using a traditional database, this application uses JSON files as a temporary data storage system. The UserStorage class is responsible for storing and retrieving user data (name, email, password) as JSONObject entries.

This method allows us to simulate a lightweight database during development and testing phases.